

INFO-F-422 - Statistical foundations of machine learning Project 2025-2026

Gianluca Bontempi, Cédric Simar, Pascal Tribel

March 31, 2026

The project counts for 50% of your final grade (i.e., 10/20). This project has to be developed by a team of 3 students registered for the class. Projects submitted by teams with a different number of students will not be considered unless prior authorization is explicitly granted. The project shall be completed independently and it shall represent the sole efforts of the team submitting the assignment. The result of another team's efforts, or the copy of another team's efforts (current, or past, semester(s)), is considered academic dishonesty and will be punished accordingly.

1 Background

System identification is concerned with learning models of dynamic systems from measured data. This project will focus on the identification of discrete-time Multi-Input Multi-Output (MIMO) Nonlinear AutoRegressive eXogenous (NARX) models. NARX models are widely used in engineering, control, signal processing, economics to describe input/output dynamics.

Many modern systems, from drones and robots to renewable-energy devices and biomedical monitors, behave in nonlinear ways and depend heavily on their past states. Traditional modeling often falls short when the physics is too complex, too expensive to derive, or simply unknown, but NARX models let engineers learn these dynamics directly from data, capturing memory effects and nonlinear relationships that matter in real operation. This capability translates into safer autonomous vehicles that can anticipate difficult driving conditions, smarter energy systems that adapt to fluctuating demand, and more reliable industrial machines that can predict faults before they happen. By understanding how to build these models, students may gain a powerful tool for turning raw data into actionable insight, preparing them to design systems that are not only intelligent but also robust in the face of real-world complexity.

A NARX with two outputs ($y_1 \in \mathbb{R}$ and $y_2 \in \mathbb{R}$) and two inputs ($u_1 \in \mathbb{R}$ and $u_2 \in \mathbb{R}$) can be described by

$$\begin{aligned} y_1(k+1) &= f_1\left(y_1(k-d), y_1(k-d-1), \dots, y_1(k-d-n_a), y_2(k-d), y_2(k-d-1), y_2(k-d-n_a), \right. \\ &\quad \left. u_1(k), \dots, u_1(k-n_b), u_2(k), \dots, u_2(k-n_b)\right) + w_1(k+1) \\ y_2(k+1) &= f_2\left(y_1(k-d), y_1(k-d-1), \dots, y_1(k-d-n_a), y_2(k-d), y_2(k-d-1), y_2(k-d-n_a), \right. \\ &\quad \left. u_1(k), \dots, u_1(k-n_b), u_2(k), \dots, u_2(k-n_b)\right) + w_2(k+1) \end{aligned} \quad (1)$$

where

- the non negative integer $k \in \mathbb{Z}_{\geq 0}$ denotes the discrete-time,
- the non negative integer $d \in \mathbb{Z}_{\geq 0}$ is the delay,
- the non negative integer $n_a \in \mathbb{Z}_{\geq 0}$ is the output lag
- the non negative integer $n_b \in \mathbb{Z}_{\geq 0}$ is the input lag
- the noise signals $\mathbf{w}_1 \sim \mathcal{N}(0, \sigma^2)$ and $\mathbf{w}_2 \sim \mathcal{N}(0, \sigma^2)$ are i.i.d. and Normally distributed with zero mean.

2 Project objective

The team is supposed to design a machine learning pipeline to learn the functions f_1, f_2 as well as the parameters d, n_a, n_b on the basis of a training dataset composed of the four time series

$$u_1^{tr}(k), u_2^{tr}(k), y_1^{tr}(k), y_2^{tr}(k) \quad k \in [1, N] \quad (2)$$

The learned model will be tested on a test dataset composed of two time series

$$u_1^{ts}(k), u_2^{ts}(k) \quad k \in [1, N_{ts}] \quad (3)$$

which obviously differ from the training set.

The code of the project team must provide the predictions

$$\hat{y}_1(k), \hat{y}_2(k) \quad k \in [1, N_{ts}] \quad (4)$$

by supposing that $y_1(0) = y_2(0) = u_1(0) = u_2(0) = 0$ and that all previous values are zero, too.

2.1 Performance metrics

The accuracy of the returned prediction will be measured in terms of:

1. The *root mean squared error* (RMSE):

$$\text{RMSE} = \sqrt{\frac{1}{2N_{ts}} \sum_{k=1}^{N_{ts}} \sum_{j=1}^2 \left(y_j^{ts}(k) - \hat{y}_j(k) \right)^2}$$

2. The *normalized mean squared error* (NMSE):

$$\text{NMSE} = \frac{\sum_{k=1}^{N_{ts}} \sum_{j=1}^2 \left(y_j^{ts}(k) - \hat{y}_j(k) \right)^2}{\sum_{k=1}^{N_{ts}} \sum_{j=1}^2 \left(y_j^{ts}(k) - \bar{y}_j \right)^2}$$

where:

- $\langle y_1^{ts}(k), y_2^{ts}(k) \rangle$ is the actual output of the unknown NARX at time k
- $\langle \hat{y}_1(k), \hat{y}_2(k) \rangle$ is the predicted output vector at time k ,
- N_{ts} is the number of test observations,
- $\bar{y}_j = \frac{1}{N_{ts}} \sum_{k=1}^{N_{ts}} y_j^{ts}(k)$ is the mean of the j -th output

2.2 Data files

The data files for the project can be downloaded using the **following link**. The dataset contains the following time series

1. Utr: contains the training input series u_1 and u_2 in the form of a Numpy array of shape (1000, 2)
2. Ytr: contains the training input series y_1 and y_2 in the form of a Numpy array of shape (1000, 2)
3. Uts1: contains the first test input series u_1 and u_2 in the form of a Numpy array of shape (1000, 2)
4. Uts2: contains the second test input series u_1 and u_2 in the form of a Numpy array of shape (1000, 2)

The project team should return the predicted output series associated to both input series obtained with the NARX model learned using the training data set made of Utr and Ytr. The prediction should be returned in the form of two Numpy arrays of shape (1000, 2) called Yhat1 and Yhat2, respectively.

The accuracy of the first prediction Yhat1 can be assessed by submitting it to the competition website (Section 6).

The final ranking score (point 5 of Section 4) will only depend on the second prediction Yhat2.



3 NARX pilot examples for designing the methodology

In order to develop and assess the learning methodology the project team should use the following two pilot NARXs for which the functions f_1 and f_2 are known.

3.1 NARX1

This system has two outputs and a single input

$$\begin{aligned}y_1(k+1) &= 0.5y_2(k-1) + \sin(y_2(k)) + 0.3u(k-1) + w_1(k+1) \\y_2(k+1) &= 0.5y_1(k-1) + \sin(y_1(k)) + 0.2u(k) + w_2(k+1)\end{aligned}\tag{5}$$

The project team is free to set the initial conditions and noise variances to the desired values.

3.2 NARX2

This system has two outputs and a two inputs

$$\begin{aligned}y_1(k+1) &= \frac{y_1(k)y_1(k-1)y_1(k-2)(y_1(k-2)-1)u_2(k-1)+u_2(k)}{1+y_2(k-1)^2+y_2(k-2)^2} + w_1(k+1) \\y_2(k+1) &= \frac{y_2(k)y_2(k-1)y_2(k-2)(y_2(k-2)-1)u_1(k-1)+u_1(k)}{[1+y_1(k-1)^2+y_1(k-2)^2]} + w_2(k+1)\end{aligned}\tag{6}$$

The project team is free to set the initial conditions and the noise variances to the desired values.

Though those two pilots are to be used to design and assess the learning methodology, be aware that that the training and the test datasets in Section 2 refer to a different and unknown NARX.

4 Project tasks and evaluation

Given the described dataset, the main overall objective of the project is to learn the NARX model corresponding to the task. Specifically, the project involves the following tasks:

1. **(1 point) Simulation of the two pilots:** setup a Python simulation of the two pilots (Section 3) to generate training and test datasets to design and assess the methodology. The simulation should allow the definition of the length of the training and test series and the setting of a specific variance of the noise.
2. Design and implement an adequate machine learning strategy for learning NARX based on input/output historical datasets. In particular, the group should use the two pilots to assess the quality of the methodology. In particular this task should include the following subtasks
 - (a) **(1 point) Parameter selection approach:** description of the methodology to find the parameters d, n_a, n_b on the basis of the observed input/output dataset.
 - (b) **(1 point) MIMO and multi-step-ahead strategy selection approach:** the main challenges of the project are i) the MIMO nature of the task and 2) that the prediction is expected not for a short horizon but for a very long one. This can be particularly challenging since the learned model could use as inputs the predictions of the step before with a consequent risk of error explosion and prediction divergence

The group should sketch the possible alternatives to deal with the above issues and then design and describe the adopted methodology.



- (c) **(2 points) Baseline approach:** Select at least two (non deep learning) different regression models to model f_1 and f_2 and compare their performance. Clearly document this process in your notebook, discussing the outcomes in detail. On the basis of the output of the previous steps, create a scikit-learn Pipeline that integrates the simulation of the training and test simulation, the learning of f_1 and f_2 and the final prediction. Using visualizations and tables to illustrate your findings, and employing formulas or pseudo-code to explain the feature selection procedure, is strongly encouraged. Note that 0.75 points will depend on the quality and clarity of your documentation.
 - (d) **(2 points) Deep learning approach:** Extend the pipeline of the previous step by integrating neural network approach using the PyTorch library. **Optionally**, you can implement both approaches. Assess its performance with respect to the baseline approach and discuss the results. The use of figures, formulas, tables and pseudo-code to describe this approach is again strongly encouraged. Note that 0.75 points will depend on the quality and clarity of your documentation.
3. **(1.5 point) Deployment of the methodology on the prediction task:** the methodology proposed in the previous steps has to be deployed for the specific learning task described in Section 2. In particular the student team should make clear how the final model was chosen and learned, which strategy for MIMO and multi-step-ahead prediction was considered, how the parameters were set and provide a script to reproduce the prediction that will be eventually submitted. The use of figures, formulas, tables and pseudo-code to describe this approach is again strongly encouraged. Note that 0.5 points will depend on the quality and clarity of your documentation.
 4. **(0.5 point) Submission and ranking:** Those points will be given only if a final submission is done, the submission coincides with the output of the script returned in the previous step and the submission is readable and in the correct format.
 5. **(1 point) Ranking score:** The final score will depend on the final ranking (in terms of NMSE) of the prediction $\hat{Y}_{at2}$ among the submitted predictions (i.e.. 1 to the best one, 0 to the worst one and proportionally for the rest).

5 The team

A project team has to be composed of exactly three students registered for the class. Team composition must be submitted **by one of the group members** through a dedicated Google Form, available [here](#). The team composition has to be finalized no later than **11PM of the 5th of April 2026**. (Hard deadline!)

6 Competition and submission format

Submission format The submission must consist of a single NumPy archive file named `submission.npz` containing two NumPy arrays called `Yhat1` and `Yhat2`. Each of these arrays must have shape $(1000, 2)$, where the 1000 rows represent the 1000 time steps of the corresponding test sequence and the 2 columns represent the predicted outputs $\hat{y}_1$ and $\hat{y}_2$. The array `Yhat1` corresponds to the prediction for `Uts1` and will be assessed on the leaderboard. The array `Yhat2` corresponds to the prediction for `Uts2` and will be used for the project marks. Both arrays must be generated by the final model trained only on `Utr` and `Ytr`. The submitted file must be readable and must contain exactly these two arrays under the exact names `Yhat1` and `Yhat2`. In Python, the submission file can be created as follows:

```
1 np.savez("submission.npz", Yhat1=Yhat1, Yhat2=Yhat2)
```

Competition website We have created a dedicated [competition website](#) where each team can submit predictions associated with the first test input series and compare their performance on the leaderboard. To submit your model's predictions, first prepare your submission as described above. Then, visit the [competition website](#), click on the Submit button, authenticate using the team's credentials (which you will have received beforehand), and upload your `submission.npz` file.



Only the prediction $\hat{Y}_{at1}$ will be assessed on the leaderboard. No leaderboard will be made available for the second prediction $\hat{Y}_{at2}$. The prediction $\hat{Y}_{at2}$ will be assessed separately after the project submission and will be used for the project marks.

The same `submission.npz` file, containing both $\hat{Y}_{at1}$ and $\hat{Y}_{at2}$, must also be included in the team's Github repository as part of the final project deliverables.

7 Github

The submission of the deliverables for the project is made on a group repository on the GitHub Classroom platform. **The access link required to setup your repository on the platform will be provided once all the teams are formed.** Once you click the link, you shall either create and name your group (i.e. "Group [group number]") or join your existing group if it has already been created by your group partner.

The version control system offered by Git allows you to easily keep track of the different versions of your code and go back to a previous version of your code in case you have any problems. In order to do so, you will need to regularly instruct the system to keep track of the different versions of your code (i.e. `commit` in the Git terminology). However, a `commit` only keeps track of a version of your code locally (on your PC). In order to transfer a copy of your code to a remote server (like Github) you will need to perform a `push` operation. We encourage you to regularly perform `commit` and `push` operations in order to have up-to-date versions of your code both locally and remotely. More information concerning the Git workflow and how to manage your repositories can be found at <https://docs.github.com/en/repositories>. A short kick-start demonstration of Git and Github Classroom will also take place during the practical session introducing the project.

For each team of students, the final submission of your code that will be evaluated is the last uploaded version of the deliverables on your assigned Github repository before the deadline time and date.

8 Students participation

The Git version control system will also be used to assess students' participation in the group effort. Students with little to no visible participation in the project will be penalized accordingly. Thus, we encourage every member of each group to regularly perform `commit` and `push` operations so that we can reliably appreciate the input of each member with respect to the final deliverables.

9 Plagiarism and code generation

Considering that new code generation tools were recently made publicly available, we would like to especially draw your attention to the fact that this project **shall be completed as a team and it shall represent your sole efforts**. Each project will be scanned for text and code plagiarism or automatic generation. The result or the copy of another team's efforts (current or past semester(s)) of a publicly available notebook, either in full or in part, or the use of an automatic code/text generation tool, is considered academic dishonesty. Plagiarism, in the sense of copy-pasting from existing or generated reports or code, is a serious issue and will lead to disciplinary measures being applied to the whole group.

10 Deliverables

The student team will deliver its assigned Github repository:

1. the documented implementation (in a Python Jupyter notebook format) of all the objectives described in this document, with a clear justification of each choice.
2. the NumPy submission file `submission.npz` containing both predictions $\hat{Y}_{at1}$ and $\hat{Y}_{at2}$, formatted as described in 6.



Rules for project submission

To be read carefully!

1. The assignment should be made by teams of **exactly** three students. The team composition has to be finalized no later than **11PM the 5th of April 2026**.
2. The assignment will be graded on the notebook containing your documented implementation
3. The code should be **commented** and each choice of implementation should be justified.
4. The assignment will be submitted through the Github Classroom platform and the repository should include all deliverables listed in the [deliverables section](#).
5. The deadline for the submission of your project is: **11PM the 5th of May 2026**.
6. All the projects submitted after the deadline will be:
 - Penalized by one point if submitted **before 11PM the 6th of May 2026**.
 - Not considered **after 11PM the 6th of May 2026**
7. Sharing of code is not allowed (you may, however, verbally discuss ideas on how to tackle the project).
8. This project counts for 50% of your grade (10 points). This project **shall be completed as a team and it shall represent your sole efforts**. The result of the copy of another team's efforts (current or past semester(s)) or automatic code generation is considered academic dishonesty. Plagiarism, in the sense of copy-pasting from existing reports or code, is a serious issue.
9. Each project producing any error during its execution will receive a grade of 0/10.

